```
import numpy as np
def cosine_similarity(a, b):
  return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
good = np.array([0.82, 0.41, 0.10])
great = np.array([0.79, 0.38, 0.10])
bad = np.array([-0.71, 0.12, -0.05])
print(f'good vs great : {cosine_similarity(good, great):.3f}')
print(f'good vs bad : {cosine_similarity(good, bad):.3f}')
print(f'great vs bad : {cosine_similarity(great, bad):.3f}')
```

```
good vs great : 1.000
good vs bad : -0.808
great vs bad : -0.817
```

```
!pip install gensim
from gensim.models import Word2Vec
# A small corpus — in practice use millions of sentences
sentences = [
'the king ruled the kingdom wisely',
'the queen ruled the land gracefully',
'the prince will become king one day',
'a man walked into the palace',
'a woman walked into the palace',
'the dog barked loudly in the garden',
'the puppy played happily in the garden',
]
# Tokenise each sentence into a list of words
tokenised = [s.split() for s in sentences]
# Train Word2Vec
# vector_size=50: each word gets a 50-number vector
# window=3: look 3 words left and right for context
# sg=1: use Skip-gram (sg=0 for CBOW)
# epochs=200: pass through the data 200 times
model = Word2Vec(sentences=tokenised, vector_size=50,
window=3, min_count=1, sg=1, epochs=200)
# Find top-3 most similar words to 'king'
print(model.wv.most_similar('king', topn=3))
# [('queen', 0.91), ('prince', 0.88), ('kingdom', 0.85)]
# The famous analogy: king - man + woman
result = model.wv.most_similar(
positive=['dog', 'puppy'], # add these
negative=['man'], # subtract this
topn=2
)
print('king - man + woman =', result[1][0]) # 'queen'
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (fro
[('kingdom', 0.40141454339027405), ('played', 0.38476690649986267), ('in', 0.3839575
king - man + woman = the
```

```
import sys
```

```
!{sys.executable} -m pip install gensim
# Step 0: Download GloVe vectors
# https://nlp.stanford.edu/data/glove.6B.zip (about 800 MB)
# Extract and you get: glove.6B.50d.txt, glove.6B.100d.txt, etc.
!wget --no-check-certificate https://nlp.stanford.edu/data/glove.6B.zip
!unzip -q glove.6B.zip

# Step 1: Convert GloVe format to Word2Vec format (one-time step)
from gensim.scripts.glove2word2vec import glove2word2vec
glove2word2vec('glove.6B.100d.txt', 'glove.6B.100d.w2v.txt')
# Step 2: Load the converted file
from gensim.models import KeyedVectors
glove = KeyedVectors.load_word2vec_format(
'glove.6B.100d.w2v.txt', binary=False
) # Takes ~30 seconds; 400,000 words x 100 dimensions
# Step 3: Use it!
# --- Find top-5 most similar words ---
print(glove.most_similar('bollywood', topn=5))
# [('cinema', 0.82), ('films', 0.79), ('actors', 0.77), ...]
# --- Test analogy ---
res = glove.most_similar(positive=['japan','andhra'], negative=['france'],
topn=1)
print('paris - france + india =', res[0][0]) # hopefully 'delhi'!
# --- Cosine similarity between two words ---
print(glove.similarity('dog', 'puppy')) # ~0.81
print(glove.similarity('cat', 'cricket')) # ~0.12 (unrelated)
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (fro
--2026-06-11 09:16:48--  https://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-06-11 09:16:48--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:44
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: ‘glove.6B.zip.3’

glove.6B.zip.3       45%[========>           ] 370.37M  5.02MB/s    eta 87s
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from gensim.models import KeyedVectors

# Assuming you already loaded GloV
glove = KeyedVectors.load_word2vec_format(
    'glove.6B.100d.w2v.txt',
    binary=False
)

# Words to visualize
words = [
    'king', 'queen', 'man', 'woman', 'prince', 'princess',
    'dog', 'puppy', 'cat', 'kitten','rama chandra','shravanth',
    'paris', 'france', 'delhi', 'india', 'london', 'england'
]

# Keep only words that exist in the model
valid_words = [w for w in words if w in glove.key_to_index]

missing_words = set(words) - set(valid_words)
if missing_words:
    print("Missing words:", missing_words)

# Extract vectors
vectors = np.array([glove[w] for w in valid_words])

print("Vector matrix shape:", vectors.shape)

# Reduce dimensions from 100D -> 2D
pca = PCA(n_components=2, random_state=42)
coords = pca.fit_transform(vectors)

# Plot
fig, ax = plt.subplots(figsize=(10, 7))

ax.scatter(
    coords[:, 0],
    coords[:, 1],
    s=80,
    color='steelblue',
    zorder=3
)

# Add labels
for i, word in enumerate(valid_words):
```

```
    ax.annotate(
        word,
        (coords[i, 0], coords[i, 1]),
        fontsize=11,
        xytext=(5, 4),
        textcoords='offset points'
    )

ax.set_title(
    'GloVe Word Clusters (PCA Projection to 2D)',
    fontsize=13
)

ax.axhline(0, color='gray', linewidth=0.5)
ax.axvline(0, color='gray', linewidth=0.5)

plt.tight_layout()
plt.savefig('word_clusters.png', dpi=150)
plt.show()
```

```
Missing words: {'shravanth', 'rama chandra'}
Vector matrix shape: (16, 100)
```